

# The **nodriver** **Cookbook**

Practical Recipes for  
Modern Browser Automation



## **Python Recipes**

Real-world automation  
with nodriver



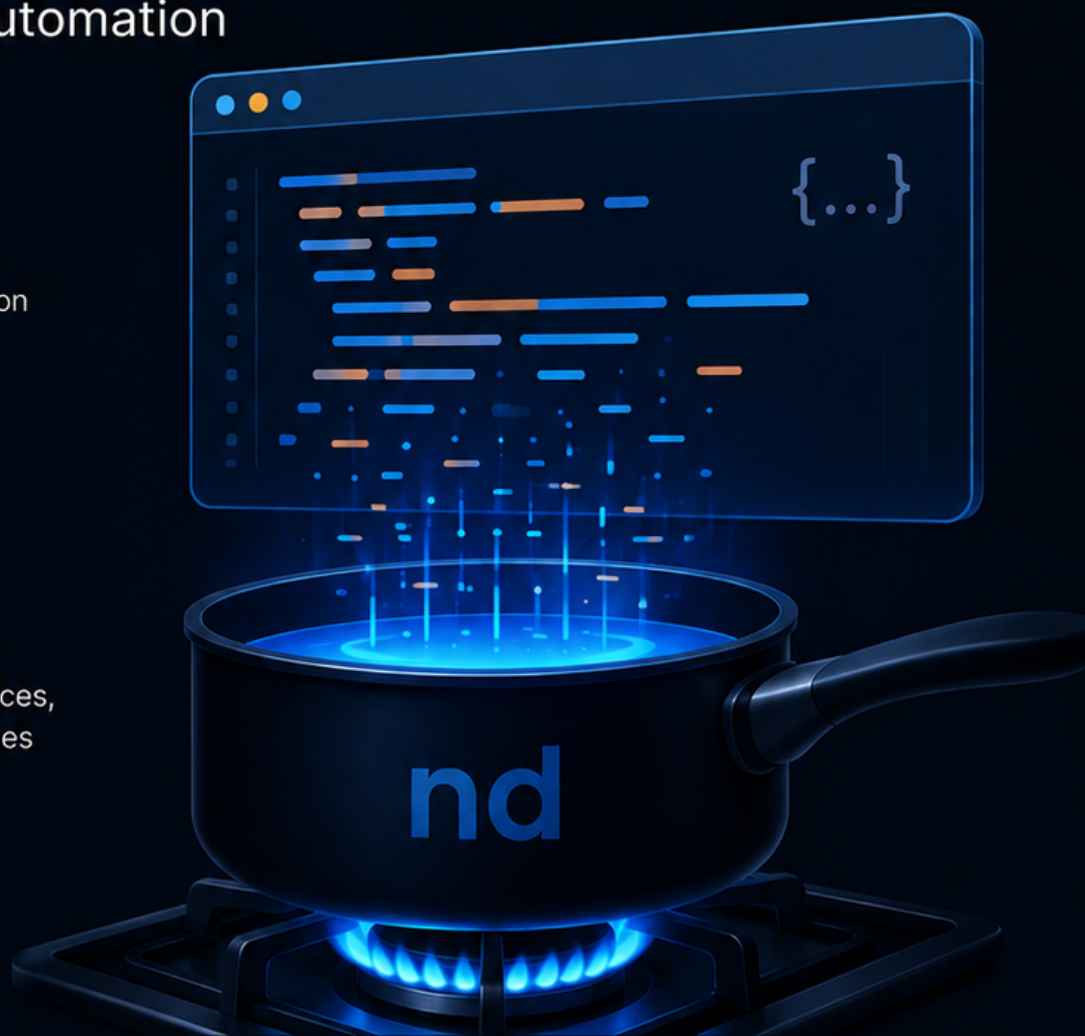
## **Modern & Async**

Built for speed,  
stability, and scale



## **Production Ready**

Patterns, best practices,  
and proven techniques



**Write Less.**  
**Automate More.**

A practical guide to building reliable,  
maintainable automations with **nodriver**.

Y A S A K A   H A N I N I

# **Python Browser Automation Cookbook**

Working Smarter with nodriver

Yasaka Hanini

July 18, 2026

# Table of contents

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>Python Browser Automation Cookbook</b>                   | <b>6</b>  |
| 1.1      | Working Smarter with nodriver . . . . .                     | 6         |
| <b>2</b> |                                                             | <b>7</b>  |
| <b>3</b> | <b>Before You Begin</b>                                     | <b>8</b>  |
| 3.1      | The Problem This Chapter Solves . . . . .                   | 8         |
| 3.2      | Required Environment . . . . .                              | 9         |
| 3.3      | Verify Python . . . . .                                     | 9         |
| 3.4      | Create Project Environment . . . . .                        | 10        |
| 3.5      | Install Dependencies . . . . .                              | 10        |
| 3.6      | Verify Chrome Installation . . . . .                        | 11        |
| 3.7      | First Test . . . . .                                        | 11        |
| 3.8      | You are ready for Recipe 1. . . . .                         | 12        |
| <b>I</b> | <b>Foundations</b>                                          | <b>13</b> |
| <b>4</b> | <b>Chapter 1: Getting Started</b>                           | <b>14</b> |
| 4.1      | The Problem This Chapter Solves . . . . .                   | 14        |
| 4.2      | Recipe 1: Launch and Close a Browser . . . . .              | 14        |
| 4.3      | Recipe 2: Open and Manage Browser Tabs . . . . .            | 16        |
| 4.4      | Recipe 3: Reuse a Browser Profile Across Sessions . . . . . | 17        |
| 4.5      | Recipe 4: Customize Browser Startup Options . . . . .       | 19        |
| <b>5</b> | <b>Chapter 2: Browser Control</b>                           | <b>21</b> |
| 5.1      | The Problem This Chapter Solves . . . . .                   | 21        |
| 5.2      | Recipe 5: Navigate Like a User . . . . .                    | 21        |
| 5.3      | Recipe 6: Capture Screenshots . . . . .                     | 22        |

|            |                                                           |           |
|------------|-----------------------------------------------------------|-----------|
| 5.4        | Recipe 7: Execute JavaScript in the Page . . . . .        | 23        |
| 5.5        | Recipe 8: Inspect Browser State . . . . .                 | 25        |
| <b>6</b>   | <b>Chapter 3: Production Foundations</b>                  | <b>27</b> |
| 6.1        | The Problem This Chapter Solves . . . . .                 | 27        |
| 6.2        | Recipe 9: Wait for Meaningful State . . . . .             | 27        |
| 6.3        | Recipe 10: Retry Transient Failures . . . . .             | 29        |
| 6.4        | Recipe 11: Add Useful Logging . . . . .                   | 31        |
| 6.5        | Recipe 12: Customize Cookbook Configuration . . . . .     | 32        |
| <b>II</b>  | <b>Data &amp; Operations</b>                              | <b>34</b> |
| <b>7</b>   | <b>Chapter 4: Elements &amp; Forms</b>                    | <b>35</b> |
| 7.1        | The Problem This Chapter Solves . . . . .                 | 35        |
| 7.2        | Recipe 13: Find Elements Using Stable Selectors . . . . . | 36        |
| 7.3        | Recipe 14: Click Elements Reliably . . . . .              | 37        |
| 7.4        | Recipe 15: Fill Forms . . . . .                           | 39        |
| 7.5        | Recipe 16: Upload Files . . . . .                         | 39        |
| 7.6        | Recipe 17: Select Dropdown Options . . . . .              | 40        |
| 7.7        | Recipe 18: Handle Dialogs and Pop-ups . . . . .           | 41        |
| <b>8</b>   | <b>Chapter 5: Authentication &amp; Sessions</b>           | <b>43</b> |
| 8.1        | The Problem This Chapter Solves . . . . .                 | 43        |
| 8.2        | Recipe 19: Log In and Verify Authentication . . . . .     | 44        |
| 8.3        | Recipe 20: Work with Cookies . . . . .                    | 45        |
| 8.4        | Recipe 21: Reuse Authenticated Sessions . . . . .         | 46        |
| 8.5        | Recipe 22: Manage Multiple Accounts . . . . .             | 47        |
| <b>9</b>   | <b>Chapter 6: Data Collection</b>                         | <b>49</b> |
| 9.1        | The Problem This Chapter Solves . . . . .                 | 49        |
| 9.2        | Recipe 23: Extract Structured Tables . . . . .            | 50        |
| 9.3        | Recipe 24: Handle Pagination . . . . .                    | 51        |
| 9.4        | Recipe 25: Scrape Infinite Scroll Pages . . . . .         | 52        |
| 9.5        | Recipe 26: Download Files Reliably . . . . .              | 53        |
| 9.6        | Recipe 27: Extract Images and Media . . . . .             | 54        |
| <b>III</b> | <b>Production</b>                                         | <b>56</b> |
| <b>10</b>  | <b>Chapter 7: Reliable Automation</b>                     | <b>57</b> |

## TABLE OF CONTENTS

---

|                                                                   |               |
|-------------------------------------------------------------------|---------------|
| 10.1 The Problem This Chapter Solves . . . . .                    | 57            |
| 10.2 Recipe 28: Build Automation That Resembles Normal Usage      | 57            |
| 10.3 Recipe 29: Build Resilient Browser Automation . . . . .      | 58            |
| <b>11 Chapter 8: Starter Kit</b>                                  | <b>60</b>     |
| 11.1 The Problem This Chapter Solves . . . . .                    | 60            |
| 11.2 Recipe 30: Production Browser Automation Starter Kit . . . . | 60            |
| <br><b>IV Production Systems</b>                                  | <br><b>63</b> |
| <br><b>12 Production Systems</b>                                  | <br><b>64</b> |
| <br><b>13 Version Compatibility</b>                               | <br><b>65</b> |
| 13.1 Compatible Versions . . . . .                                | 65            |

# Python Browser Automation Cookbook

## Working Smarter with nodriver

Version 1.0.0

Welcome to the *Python Browser Automation Cookbook*. This book provides practical, production-ready recipes for browser automation using Python and nodriver.

**Who this is for:** Developers who want to automate browser tasks reliably — from web scraping and form filling to monitoring and data extraction.

**How to use this book:** Each chapter is organized around a specific capability. Start with Chapter 0 for setup, then work through the chapters in order. Each recipe is self-contained but builds on previous chapters.

**About nodriver:** nodriver is a modern Python library for browser automation using Chrome DevTools Protocol (CDP). It is lightweight, async-native, and does not require WebDriver or Selenium.

## **Python Browser Automation Cookbook**

*Working Smarter with nodriver*

Version 1.0.0

© 2026 Yasaka Hanini. All rights reserved.

This publication is protected by copyright. No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews.

### **License**

- The book content is licensed for personal use only. Redistribution is prohibited. - Source code accompanying this book may be modified and used for personal and commercial projects. - Reselling the book or its derivative works is prohibited.

### **Version History**

| Version | Date | Notes           |  |  |  |
|---------|------|-----------------|--|--|--|
| 1.0.0   | 2026 | Initial release |  |  |  |

# Before You Begin

## The Problem This Chapter Solves

Most setup guides tell you to “pip install nodriver” and call it done. Then you hit the first error — wrong Python version, missing Chrome, permission issues, version mismatch — and you have no mental map of what’s broken.

This chapter walks through a complete, verified setup. By the end, you have launched a browser from Python and closed it successfully. Chapter 1 starts from there.

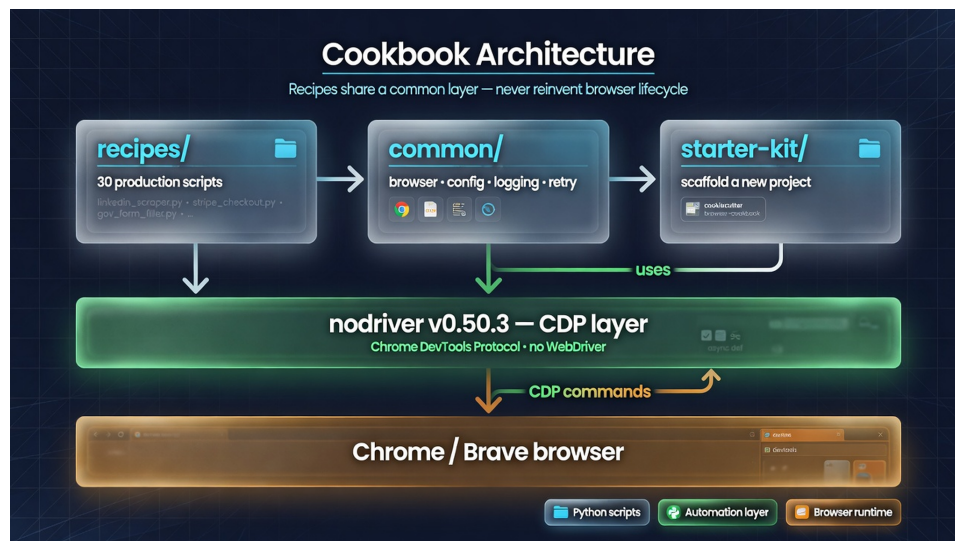


Figure 3.1: Browser Automation Architecture



## Required Environment

Your automation stack has five layers. Every layer must be present and compatible.

Your Machine (Windows / macOS / Linux)

Python 3.11+ (virtual environment)

webdriver 0.50.3 (pip package)

Chrome 130+ (browser)

Your Automation Scripts

The next five sections verify each layer.

## Verify Python

Open a terminal and run:

```
python --version
```

**Expected output:**

Python 3.11.x

**Why 3.11?** Three reasons:

1. **Async performance** — Python 3.11 optimized async task switching. Browser automation makes thousands of async calls per session. The speed difference vs 3.9 is measurable.
2. **Error messages** — 3.11+ pinpoints the exact expression that failed in a traceback. When a `page.find()` returns `None` and your code crashes on `None.text`, the traceback shows you exactly where.
3. **Typing support** — 3.11 added `Self` type and improved `TypedDict`. `webdriver`'s type hints are most accurate under 3.11+.

If you see 3.8 or lower, install Python 3.11 from [python.org](https://python.org) before proceeding.

## Create Project Environment

Never install automation dependencies globally. Browser tooling has frequent version conflicts with system packages and other projects.

```
mkdir browser-automation-project
cd browser-automation-project
python -m venv .venv
```

Activate the environment:

**Windows (cmd/PowerShell):**

```
.venv\Scripts\activate
```

**Windows (Git Bash):**

```
source .venv/Scripts/activate
```

**macOS / Linux:**

```
source .venv/bin/activate
```

Your prompt should now show `(.venv)` at the beginning.

## Install Dependencies

Upgrade pip first. Pip versions older than 22 can silently install wrong versions.

```
pip install --upgrade pip
pip install nodriver==0.50.3
```

Pin the version in a requirements file. This ensures reproducible installs across machines and deployments.

```
pip freeze > requirements.txt
```

**Why pin nodriver?** Browser automation is sensitive to three things that change between versions:

- **Chromium protocol changes** — Chrome updates the CDP protocol frequently. nodriver must match.

- **API surface changes** — What worked in 0.48 may not work in 0.50.
- **Bug fixes** — A patch version may change behavior of `find()`, `click()`, or `send_keys()`.

A `requirements.txt` with pinned versions is your insurance against “it worked yesterday.”

## Verify Chrome Installation

`nodriver` controls Chrome. It does not replace Chrome.

**Windows:** Open Chrome and go to `chrome://version`. Note the version number.

**macOS / Linux:**

```
google-chrome --version
# or
chromium-browser --version
```

**Expected:** Version 130 or higher.

**What if Chrome is not installed?** Download from [google.com/chrome](https://www.google.com/chrome). Chromium (the open-source base) also works.

**What about Brave, Edge, or other Chromium browsers?** They work. Specify the executable path when calling `launch_browser()`. The cookbook uses Chrome by default.

## First Test

Create a small verification script before running any recipes.

Create `test_browser.py`:

```
import asyncio
import nodriver as uc

async def main():
    browser = await uc.start()
    page = await browser.get("https://example.com")
    title = await page.title()
    print(f"Chrome opened. Page: {title}")
```

```
browser.stop()

if __name__ == "__main__":
    asyncio.run(main())
```

Run it:

```
python test_browser.py
```

**Expected output:**

Chrome opened. Page: Example Domain

**What to check for:**

---

| Observation                           | Meaning                                        |
|---------------------------------------|------------------------------------------------|
| Chrome window appears and closes      | Browser works, CDP connected                   |
| Error “Chrome not found”              | Chrome path missing                            |
| Error “Connection refused”            | Port conflict, close other<br>Chrome instances |
| Error “ModuleNotFoundError: nodriver” | Environment not activated or<br>not installed  |

---

**If the test fails:** Fix the issue now. Every recipe in this book assumes a working browser connection. Debugging a broken environment while learning new concepts is frustrating and unnecessary.

## You are ready for Recipe 1.

**Part I**

**Foundations**

# Chapter 1: Getting Started

## The Problem This Chapter Solves

Browser automation fails most often because developers treat the browser as a simple object. It is not. It is a separate operating system process that speaks a wire protocol. Launching it, managing its lifecycle, and reusing its state across runs are not “setup steps.” They are the foundation your automation reliability depends on.

This chapter covers four recipes. Each solves one piece of the browser lifecycle:

1. Launch and close — the minimal loop
2. Tabs — working with multiple pages
3. Profiles — persisting state across runs
4. Configuration — controlling how the browser starts

## Recipe 1: Launch and Close a Browser

**File:** `recipes/01_launch_browser.py`

### Why This Matters

Your Python code controls a **separate Chrome process** through a **Web-Socket connection**. This is not a library calling internal functions. It is an inter-process communication channel. If you do not understand this, every timeout error and connection refused will look like a bug when it is actually a lifecycle misunderstanding.

The minimal loop — launch, navigate, close — is the heartbeat of every automation project. Get this right and everything else follows.

## The Code

```
import asyncio
from common.browser import launch_browser, close_browser

async def main():
    browser = await launch_browser()
    page = await browser.get("https://example.com")
    title = await page.title()
    print(f"Page title: {title}")
    await close_browser(browser)

if __name__ == "__main__":
    asyncio.run(main())
```

## Code Walkthrough

`launch_browser()` starts Chrome with a temporary profile and connects via CDP. Every call gets a clean session with no leftover state from previous runs.

`browser.get()` navigates to a URL and returns an object representing the page. It waits for the page's load event before returning.

`await page.title()` retrieves the document title. This is a simple CDP script evaluation.

`close_browser()` terminates the browser process. If you forget this, Chrome stays running in the background.

## Decision Table

| Goal                  | Fresh launch             | Reuse profile            |
|-----------------------|--------------------------|--------------------------|
| One-off scrape        | <input type="checkbox"/> | <input type="checkbox"/> |
| Run every hour        | <input type="checkbox"/> | <input type="checkbox"/> |
| Test environment      | <input type="checkbox"/> | <input type="checkbox"/> |
| Authenticated session | <input type="checkbox"/> | <input type="checkbox"/> |